TaskFlow Project Blueprint

📋 Project Blueprint — TaskFlow (Task Manager App)

Purpose of this document: This is our shared source of truth. Before writing a single line of code, all three of us read this and agree on it. If anything here needs to change later, we update this file first, then commit the change — not the other way around.

---

1. Project Overview

Project Name: TaskFlow **What it does:** A task manager where users register/login and manage their personal to-do list (create, update, delete, mark complete, filter by status/priority/due date). **Why this project:** Small enough to actually finish, but has real auth + real CRUD + real integration points — enough to practice professional Git collaboration properly.

Tech Stack - Backend: Node.js + Express - Database: MongoDB (Atlas) - Frontend: React - Auth: JWT (JSON Web Tokens) - Deployment: Backend → Railway/Render, Frontend → Vercel - CI: GitHub Actions

---

2. Team & Roles

Name	Git Role	Ownership
Saurabh	Dev A — Backend / Core API	Task management logic end-to-end
Gaurav	Dev B — Frontend	Everything the user sees and clicks
Sumit	Dev C — Auth & Integration/DevOps	Identity system + glue holding A and B together

Saurabh (Dev A) — Backend / Core API Developer

- Task CRUD endpoints (create, read, update, delete, mark complete)
- Task database schema
- Business logic: validation, filtering, sorting, due dates, priority
- Wiring task routes through the auth middleware Sumit builds

Gaurav (Dev B) — Frontend Developer

- Login/Register pages
- Task dashboard, task cards, task forms
- API integration layer (calling both the auth API and the task API)
- Client-side auth state/context

Sumit (Dev C) — Auth & Integration/DevOps Developer

- Auth system: register, login, JWT issuing/verification
- `verifyToken` middleware — used by Saurabh's routes
- Environment configs, `.env` management, Docker setup
- CI pipeline (GitHub Actions) and the actual deployment

Why this split matters: Sumit sits in the middle. Saurabh's routes literally cannot run without Sumit's middleware, and Gaurav's login page can't work without Sumit's auth API. This is intentional — it forces real coordination instead of three people working in silos, which is exactly what real teams deal with.

---

3. Folder Structure

```
taskflow/
├── backend/
│   ├── src/
│   │   ├── models/          # User.js (Sumit), Task.js (Saurabh)
│   │   ├── routes/          # auth.routes.js (Sumit), task.routes.js (Saurabh)
│   │   ├── controllers/     # authController (Sumit), taskController (Saurabh)
│   │   ├── middleware/      # verifyToken.js — Sumit owns, Saurabh imports
│   │   └── config/          # db.js, env loading — Sumit
│   ├── package.json
│   └── .env.example
```

```
├── frontend/
│   ├── src/
│   │   ├── pages/          # Login, Register, Dashboard — Gaurav
│   │   ├── components/     # TaskCard, TaskForm, Navbar — Gaurav
│   │   ├── services/       # api.js — Gaurav, built against the contract below
│   │   └── context/        # AuthContext.js — Gaurav, shaped by Sumit's contract
│   └── package.json
├── .github/
│   └── workflows/          # ci.yml — Sumit
├── docs/
│   └── API_CONTRACT.md     # shared — all three read/edit this before changing endpoints
├── docker-compose.yml      # Sumit
├── README.md
└── .gitignore
```

---

4. Database Design

User (owned by Sumit) | Field | Type | Notes | |—|—|—| | id | ObjectId | Primary key | | name | String | | | email | String | Unique | | passwordHash | String | Never store plain text | | createdAt | Date | |

Task (owned by Saurabh) | Field | Type | Notes | |—|—|—| | id | ObjectId | Primary key | | title | String | Required | | description | String | Optional | | status | Enum | `todo` / `in-progress` / `done` | | priority | Enum | `low` / `medium` / `high` | | dueDate | Date | Optional | | userId | ObjectId | Foreign key → User (Sumit's model) | | createdAt / updatedAt | Date | |

---

5. API Contract

This is the agreement between all three of us. **Nobody changes a request/response shape without updating this table and telling the other two.**

Endpoint	Method	Owner	Auth required?	Purpose
`/api/auth/register`	POST	Sumit	No	Create account
`/api/auth/login`	POST	Sumit	No	Get JWT
`/api/auth/me`	GET	Sumit	Yes	Get current user
`/api/tasks`	GET, POST	Saurabh	Yes	List / create tasks
`/api/tasks/:id`	GET, PUT, DELETE	Saurabh	Yes	Single task ops
`/api/tasks/:id/status`	PATCH	Saurabh	Yes	Update status only

Why this exists: Gaurav needs to build the login form before Sumit's auth API is finished, and Saurabh needs to build task routes before Sumit's middleware is finished. The contract lets everyone work in parallel against an agreed shape, using mock data, instead of waiting on each other.

---

6. Where Our Work Will Intersect (future coordination/conflict points)

- **Saurabh ↔ Sumit:** Task routes depend on `verifyToken` middleware. Saurabh will mock it on Day 1, then swap in Sumit's real version once it's ready.
- **Gaurav ↔ Sumit:** Login/Register pages depend on the real auth API. Gaurav will mock responses first.
- **Saurabh ↔ Gaurav:** Dashboard is built against the task API contract, possibly before Saurabh's endpoints are fully live.

These intersection points are exactly where we'll practice real Git skills: pull requests, code review, and resolving merge conflicts — not avoiding them, but knowing how to handle them calmly.

---

7. How We'll Work — GitHub Contribution Flow (Basic Idea)

This is the mental model all three of us should have before touching the repo. It's the same loop every professional team repeats, day after day:

```
1. PULL     ->  Get the latest version of main onto your machine
2. BRANCH   ->  Create a new branch for the one thing you're building
3. CODE     ->  Make changes, save, test locally
4. COMMIT   ->  Save a checkpoint of your changes with a clear message
5. PUSH     ->  Send your branch up to GitHub
6. PR       ->  Open a Pull Request asking to merge your branch into main
```

```
7. REVIEW   ->  The other two look at your code, comment, approve
8. MERGE    ->  Your branch's changes join main
9. REPEAT   ->  Everyone pulls the updated main, and the loop starts again
```

Why it's a loop, not a straight line: `main` is constantly moving forward as people merge in. So step 1 (pull) always happens again before you start new work — you never want to branch off an outdated version of main.

Step-by-step, in plain terms

1. **Pull** — `git pull origin main`. Downloads whatever the other two have already merged, so you start from the current real state of the project, not yesterday's.
2. **Branch** — `git checkout -b feature/task-crud-api`. A branch is your own isolated workspace — a copy of the codebase where you can experiment without touching what Gaurav or Sumit are doing.
3. **Code** — do the actual work for *one* feature. Not five features in one branch — one clear purpose per branch.
4. **Commit** — `git add .` then `git commit -m "add task creation endpoint"`. A commit is a saved snapshot with a message explaining *why*, not just *what*.
5. **Push** — `git push origin feature/task-crud-api`. Uploads your branch to GitHub — it does **not** touch `main` yet.
6. **Pull Request (PR)** — on GitHub, you formally ask: "merge `feature/task-crud-api` into `main`." This is where the team sees your diff before it becomes part of the real project.
7. **Review** — the other two read your code, leave comments, request changes if needed. Catches bugs and keeps everyone aware of what's changing.
8. **Merge** — once approved, the PR is merged. Your code is now officially part of `main`.
9. **Sync up** — everyone runs `git pull origin main` again to get each other's merged work before starting their next branch.

Rules that come out of this flow

- `main` should always be in a working state — nobody merges broken code.
- One branch = one feature/fix. Keeps PRs small and reviewable.
- Every PR gets read by at least one teammate before merging — no self-merging.
- Pull before you branch, every single time.

The overall phase plan

1. **GitHub setup** — one shared repo, all three added as collaborators, `main` branch protected (no direct pushes).
2. **Branching strategy** — short-lived feature branches per task (e.g. `feature/task-crud-api`, `feature/login-page`, `feature/jwt-auth`), never long-lived personal branches.
3. **Daily workflow** — the pull -> branch -> code -> commit -> push -> PR loop above, repeated daily.
4. **Code review** — the other two review every PR before merge. No self-merging.
5. **Merge conflicts** — when Saurabh and Sumit both touch `middleware/verifyToken.js`, we resolve it together and learn from it, not fear it.
6. **Professional practices** — clear commit messages, descriptive PR templates, GitHub Issues for tracking tasks, milestones for phases.
7. **Deployment** — backend to Railway/Render, frontend to Vercel, with CI running tests/checks before merge is even allowed.

---

8. Ground Rules (all three agree to these)

- No one pushes directly to `main`. Ever.
- Every change goes through a Pull Request, reviewed by at least one other teammate.
- If your work depends on someone else's unfinished piece, mock it — don't wait idle.
- If the API contract needs to change, update `docs/API_CONTRACT.md` in the same PR and flag it to the other two.
- Small, frequent commits with clear messages beat one giant commit at the end.

This blueprint is a living document. Update it as decisions change — but always as a tracked commit, never a silent edit.